클라이언트 구현 포트폴리오

C++ DirectX 11 멀티플레이 던전 RPG 클라이언트

저자
배진후

1 프로젝트 개요

표 1: 프로젝트 요약

항목	내용
프로젝트명	Isometric Client — DX11 멀티플레이 던전 RPG
기술 스택	C++, DirectX 11, Protobuf 3, Dear ImGui, WinSock2
서버	ServerCore v4 기반 GameServer (WSL2 Linux)
프로토콜	Binary Header `[size:uint16][msgId:uint16]` + Protobuf 3 payload
핵심 기술	인스턴스드 던전 렌더링, 프로시저럴 이펙트 셰이더, 비동기 네트워킹, 씬 기반 상태 관리

DirectX 11 API로 제작한 멀티플레이 던전 RPG 클라이언트입니다. ServerCore 포트폴리오에서 구현한 io_uring 기반 게임서버와 연동되며, 엔진 런타임에 의존하지 않고 DX11 파이프라인, 리소스 로더, 네트워크 계층을 직접 구현해 저수준 렌더링과 네트워크 동작을 다뤄보는 것이 목표였습니다.

```
isometric_client/
├── src/
│   ├── Core/       App, DX11Device, EngineContext, Input, Timer, WinMain
│   ├── Game/       PlayerController, EntityManager, CombatManager,
│   │               DungeonGenerator, SessionState
│   ├── Renderer/   Pipeline, Camera, MeshCache, MaterialManager, ObjLoader,
│   │               InstanceRenderer, EffectRenderer, MinimapRenderer,
│   │               TextureLoader, Material
│   ├── Network/    TcpClient, PacketFramer, PacketHandler, PacketBuilder,
│   │               RecvBuffer, NetworkContext
│   ├── Scene/      IScene, LoginScene, CharSelectScene, LobbyScene, GameScene
│   ├── UI/         UIManager
│   └── Data/       PlayerData, SkillData, InventoryData, CurrencyData, ChatHistory
├── assets/
│   ├── models/     29개 OBJ + MTL (Kenney 던전 에셋 + 캐릭터)
│   ├── shaders/    default.hlsl, effect.hlsl, minimap.hlsl
│   └── textures/   캐릭터·던전 텍스처
└── proto/          7개 Protobuf 정의 (59 메시지)
```

2 아키텍처와 실행 흐름

2.1 시스템 레이어

`App&`을 세 가지 Context 구조체(`EngineContext`, `SessionState`, `NetworkContext`)로 분리해 각 레이어가 필요한 데이터와 서비스를 명확히 구분하도록 설계했습니다.

표 2: Context 분리 구조

구조체	내용	역할
EngineContext	DX11Device, Pipeline, Camera, MeshCache, MaterialManager, Input, UIManager, assetsDir, pipelineOk	재사용 가능한 엔진 인프라 포인터 번들
SessionState	PlayerData, InventoryData, CurrencyData, ChatHistory, playerName, playerId, isLocalPlayerDead, MapData 캐시, pendingEnterGame	씬 전환을 넘어 유지되는 게임 상태
NetworkContext	TcpClient, PacketHandler, PacketBuilder + Connect/Disconnect/SendSceneReady API	네트워크 프로토콜 계층 + 서버 주도 흐름 제어

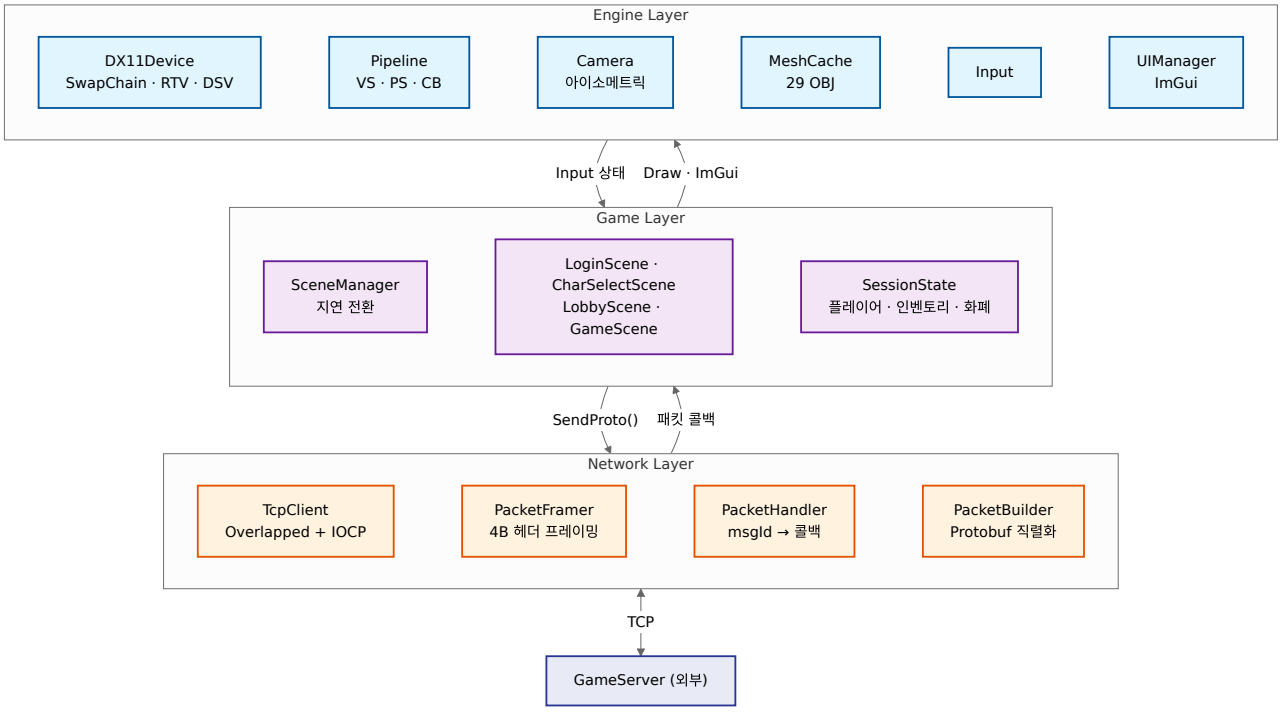

그림 1: 3-Layer 아키텍처와 레이어 간 데이터 흐름

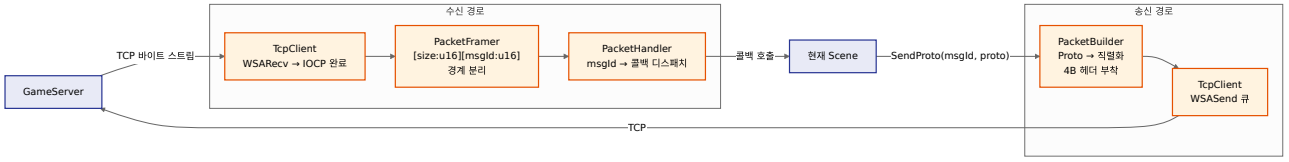

그림 2: Network Layer 내부 — 패킷 송수신 파이프라인

`SessionState`는 단순한 데이터 컨테이너가 아니라 Proto↔로컬 변환 헬퍼(`FillLocalPlayerFromProto`)와 전체 초기화(`Reset`)를 포함합니다. 연결 끊김 시 `Reset()`이 모든 서브시스템(Inventory, Currency, ChatHistory 등)을 일괄 초기화하여 재접속 시 이전 상태가 잔류하지 않도록 합니다.

`NetworkContext`는 `Connect(host, port)` / `Disconnect()` / `IsConnected()` API를 제공해 씬에서 직접 소켓 객체를 다루지 않도록 캡슐화합니다.

Scene 팩토리에서 세 컨텍스트와 SceneManager 참조를 전달합니다.

```
App.cpp — Scene 팩토리 등록
```

```
sceneManager_.Register(SceneId::Game, [this]() {
    return std::make_unique<GameScene>(engineCtx_, session_, netCtx_, sceneManager_);
});
```

2.2 메인 루프

`WinMain`의 메시지 루프가 매 프레임 다음 순서를 반복합니다. 네트워크 수신은 `App::PollNetwork()`가 IOCP의 완료 큐를 0 ms 타임아웃으로 드레인하면서 처리하므로, 게임 업데이트 직전에 한 번만 호출하면 한 프레임에 도착한 모든 패킷이 한꺼번에 디스패치됩니다.

```
Input.NewFrame → PeekMessage/DispatchMessage → App.PollNetwork() → Timer.Tick(dt) →
App.Update(dt) → App.Render()
```

`App::Update(dt)`는 `SceneManager::Update(dt)`를 호출하고, `App::Render()`는 `BeginFrame() → Scene.OnRender() → UIManager.BeginFrame() → Scene.OnUI() → UIManager.EndFrame() → EndFrame()` 순서로 처리합니다.

2.3 씬 전환과 패킷 버퍼링

`SceneManager::ChangeScene()`은 호출 시점에 즉시 전환하지 않고, `hasPending_` 플래그만 세워 두었다가 다음 `Update()` 호출에서 실제 전환을 수행합니다. 패킷 핸들러 콜백 내부에서 `ChangeScene()`을 호출하면, 전환된 새 Scene이 현재 콜백 실행 중인 이전 Scene의 핸들러를 문제가 발생할 수 있기 때문입니다. 콜백이 완전히 반환된 뒤에만 Scene이 교체됩니다.

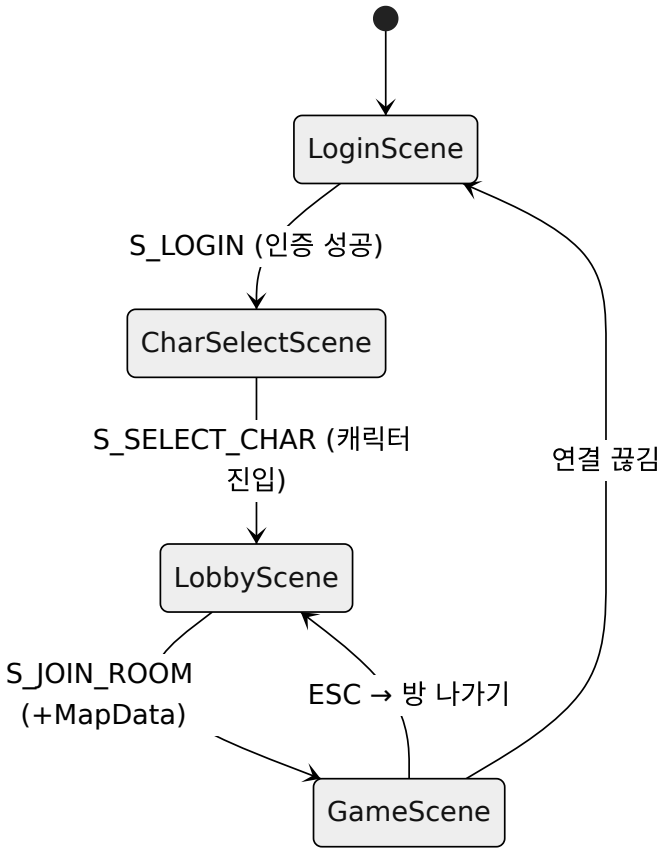

그림 3: 4단계 씬 흐름과 패킷 교환

표 3: 씬별 역할과 패킷

씬	역할	주요 패킷
LoginScene	회원가입·로그인	C/S_LOGIN, C/S_REGISTER
CharSelectScene	캐릭터 목록·생성·선택	C/S_CHAR_LIST, C/S_CREATE_CHAR, C/S_SELECT_CHAR
LobbyScene	방 목록·생성·참가	C/S_ROOM_LIST, C/S_CREATE_ROOM, C/S_JOIN_ROOM
GameScene	게임 플레이 (26종 핸들러)	이동·전투·소셜·인벤토리·화폐·루팅

3 렌더링 파이프라인

3.1 셰이더 구성과 라이팅

`Pipeline` 클래스가 `default.hlsl` 에서 VS/PS를 컴파일하고, 2개의 Constant Buffer를 관리합니다.

표 4: 셰이더 상수 버퍼 구성

상수 버퍼	슬롯	내용
PerFrame	`b0`	ViewProj 행렬, LightDir, AmbientColor
PerObject	`b1`	World 행렬, HitFlash (0~1)

Pixel Shader는 Lambert 디퓨즈 라이팅에 `HitFlash` 파라미터를 결합해, 피격 시 빨간색 플래시 효과를 표현합니다.

```
default.hlsl — Pixel Shader
float4 PSMain(VS_OUT i) : SV_Target
{
    float3 n = normalize(i.WorldNormal);
    float ndotl = saturate(dot(n, -LightDir));
    float4 tex = DiffuseTex.Sample(LinearSampler, i.UV);
    float3 lit = tex.rgb * (AmbientColor + ndotl * 0.7);
    float3 hitColor = float3(1.0, 0.3, 0.2);
    lit = lerp(lit, hitColor, HitFlash);
    return float4(lit, 1.0);
}
```

3.2 인스턴스드 던전 렌더링

던전은 많은 수의 바닥과 벽 타일로 구성됩니다. 측정에 사용한 던전은 인스턴스 4640개로 구성되어 있으며, 초기 구현에서는 타일마다 개별 `DrawIndexed()` 를 호출했습니다. 타일 한 개를 그릴 때마다 `IASetVertexBuffers` / `IASetIndexBuffer` / `UpdatePerObject` (Map/Unmap+memcpy) + `DrawIndexed` 가 반복되며 CPU→GPU 커맨드 제출이 병목이 되었습니다. 대부분의 타일은 floor, wall 등 8종 정도의 메시만 공유하므로, 같은 메시를 사용하는 타일을 하나의 Draw Call로 묶을 수 있었습니다. `InstanceRenderer` 가 메시 이름 기준으로 인스턴스를 그룹화하고, Structured Buffer에 World Matrix 배열을 적재해 메시 종류당 1 Draw Call로 처리합니다.

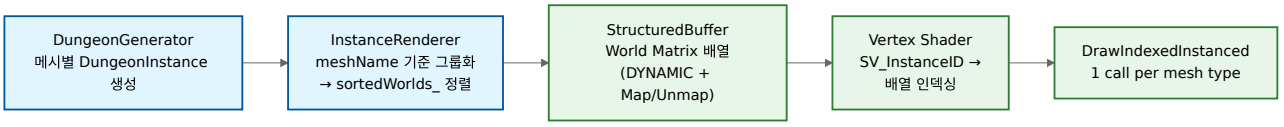

그림 4: 인스턴스드 렌더링 파이프라인

표 5: 인스턴스 렌더링 구성 요소

구성 요소	역할
DungeonInstance	메시 이름 + World Matrix 쌍
BatchInfo	메시별 `{startIndex, count}` — Structured Buffer 내 구간
EnsureBuffer()	인스턴스 수 초과 시 버퍼 재할당 (capacity 부족 방지)
BindSingle()	캐릭터 등 단일 엔티티용 1-element 버퍼 경로

인스턴스 수가 현재 capacity를 초과하면 1.5배로 재할당합니다. 최소 256개로 할당해 작은 던전에서도 버퍼 부족이 발생하지 않도록 했습니다.

```
InstanceRenderer.cpp — EnsureBuffer()
uint32_t newCapacity = (neededCount < 256) ? 256 : ((neededCount * 3) / 2);

D3D11_BUFFER_DESC bd = {};
bd.ByteWidth = newCapacity * sizeof(XMFLOAT4X4);
bd.Usage = D3D11_USAGE_DYNAMIC;
bd.BindFlags = D3D11_BIND_SHADER_RESOURCE;
bd.MiscFlags = D3D11_RESOURCE_MISC_BUFFER_STRUCTURED;
bd.StructureByteStride = sizeof(XMFLOAT4X4);
```

`InstanceRenderer`는 두 개의 Structured Buffer를 운용합니다. 배치 렌더링용 `instanceBuffer_`(256+ 용량, 1.5배 지수 성장)와 단일 엔티티용 `singleBuffer_`(1-element, 매 프레임 Map/Unmap)를 분리했습니다. 캐릭터처럼 단일 엔티티를 렌더링하려면 인스턴싱 셰이더와 별도의 non-instanced 셰이더를 유지해야 하는데, 파이프라인이 분기되면 셰이더 상태 전환과 코드 중복이 발생합니다. 1-element Structured Buffer를 두면 동일한 인스턴싱 셰이더로 단일 엔티티도 처리할 수 있어 렌더링 경로를 하나로 통일했습니다.

3.2.1 측정

성능 비교를 위해 GameScene에 `useInstancing_` 토글과 D3D11 Timestamp Query 기반 측정 코드를 잠시 도입했습니다. 두 경로(`DrawIndexed` × N vs `DrawIndexedInstanced` × M)를 런타임에 전환하면서 던전 렌더링 블록만 감싸 CPU 시간(`std::chrono::high_resolution_clock`)과 GPU 시간(D3D11 Timestamp Query, 3-frame latency)을 동시에 측정했습니다.

표 6: 인스턴싱 방식별 성능 비교 (4640 인스턴스)

모드	Draw Call	CPU (µs)	GPU (µs)	비고
Per-instance (`DrawIndexed` × N)	**4,640**	**7,936.5**	**3,585.9**	기준선
Instanced (`DrawIndexedInstanced` × M)	**8**	**34.8**	**80.3**	메시 종류 수 = 8

CPU time이 단축된 것이 가장 큰 효과입니다. 병목이었던 Draw Call 수가 극적으로 줄어든 덕분에, 4640번의 `DrawIndexed`마다 반복되던 IA 상태 변경, PerObject 상수 버퍼 Map/Unmap, 드라이버 검증 비용이 메시 종류당 1회로 축소되었습니다. 한 프레임의 16ms budget 중 던전 렌더링이 차지하던 약 8ms가 35µs로 줄어 다른 시스템(전투 효과, UI, 네트워크 처리)에 여유 시간을 확보할 수 있었습니다.

3.3 이펙트 셰이더

텍스쳐 기반 이펙트 대신, `effect.hlsl`에서 코드로 이펙트를 생성합니다.

기본 공격(좌클릭) 흐름은 발사체 → 적중 두 단계로 구성됩니다. `CombatManager::HandleFire()`가 `Projectile`을 생성해 매 프레임 직선 이동·벽 충돌을 처리하고, `HandleDamage()`가 적중 시 `HitEffect`를 타겟 위치에 등록합니다. 이 두 종류가 `EffectRenderer`를 거쳐 빌보드 쿼드로 렌더링되며, 셰이더의 `EffectType` 값으로 분기됩니다.

표 7: 기본 공격에 사용되는 프로시저럴 이펙트

EffectType	용도	셰이더 기법
3 (Bullet)	발사체 본체	코어 + 방사형 글로우
4 (Hit)	적중 순간	확장 링 + 중앙 플래시

`EffectRenderer`는 카메라의 Right/Up 벡터로 빌보드 쿼드를 구성합니다. 버텍스 셰이더는 월드 좌표 변환만 수행하고, 모든 형태는 픽셀 셰이더의 UV 기반 수식으로 만들어집니다. `EffectTime`(0→1 정규화 수명)이 알파에 직접 곱해져 자동 페이드 아웃됩니다.

```
effect.hlsl — 빌보드 정점 변환
float3 worldPos = EffectPos
    + CameraRight * i.Pos.x * EffectScale
    + CameraUp    * i.Pos.y * EffectScale;
```

Bullet — 중앙 코어, 방사형 글로우, 그리고 발사 방향 반대쪽으로 늘어지는 트레일을 합성합니다. 트레일은 `centered.x`(-0.5~+0.5)에 대한 비대칭 `smoothstep`으로 만들어, 한 방향으로만 늘어진 모양이 자연스럽게 표현됩니다.

```
effect.hlsl — Bullet
float4 Bullet(float2 uv, float t)
{
    float2 centered = uv - 0.5;
    float d = length(centered) * 2.0;

    float core  = 1.0 - smoothstep(0.0, 0.3, d);
    float glow  = 1.0 - smoothstep(0.1, 0.8, d);
    float trail = saturate(1.0 - smoothstep(-0.5, 0.1, centered.x))
                * smoothstep(0.3, 0.0, abs(centered.y));

    float3 col = float3(1,1,0.9) * core
               + float3(1,0.7,0.2) * glow * 0.6
               + float3(1,0.5,0.1) * trail * 0.4;
    float alpha = saturate(core + glow * 0.5 + trail * 0.3);
    return float4(col, alpha);
}
```

Hit — 적중 순간의 시각 강조를 위해 세 개의 레이어를 합성합니다. 시간에 비례해 반지름이 커지는 링(`ringRadius = t·0.8`), 빠르게 사라지는 중앙 플래시(`saturate(1 - t·4)`), 그리고 6방향으로 뻗어 나가는 스파크입니다. 스파크는 픽셀의 `atan2(y, x)` 각도를 6등분(`i·π/3`)한 기준 각도와 비교해 가까운 픽셀만 활성화합니다.

```
effect.hlsl — Hit
float4 Hit(float2 uv, float t)
{
    float2 centered = uv - 0.5;
```

```
    float d = length(centered) * 2.0;

    float ringRadius = t * 0.8;
    float ring = smoothstep(ringRadius - 0.15, ringRadius, d)
               * smoothstep(ringRadius + 0.15, ringRadius, d);
    float flash = (1.0 - smoothstep(0.0, 0.4, d)) * saturate(1.0 - t * 4.0);

    float angle = atan2(centered.y, centered.x);
    float sparks = 0.0;
    for (int i = 0; i < 6; i++) {
        float a = (float)i * 1.0472;  // 2π/6
        float sparkD = abs(angle - a);
        sparkD = min(sparkD, 6.2832 - sparkD);
        float sparkRadius = t * 0.7;
        sparks += smoothstep(0.15, 0.0, sparkD)
                * smoothstep(sparkRadius - 0.1, sparkRadius, d)
                * smoothstep(sparkRadius + 0.1, sparkRadius, d);
    }

    float fadeOut = 1.0 - t;
    float3 col = float3(1,0.6,0.2) * ring
               + float3(1,1,0.8) * flash
               + float3(1,0.4,0.1) * sparks * 0.5;
    return float4(col, saturate(ring + flash + sparks * 0.5) * fadeOut);
}
```

투명 파티클이 서로를 가리지 않도록 깊이 쓰기를 비활성화하고(`DepthWriteMask = ZERO`), 가산 블렌딩(`SrcBlend = SRC_ALPHA`, `DestBlend = ONE`)으로 겹칠수록 밝아지는 효과를 만듭니다. 렌더링 전후로 이전 BlendState와 DepthStencilState를 저장·복원해 다른 렌더링 경로에 영향을 주지 않습니다.

3.4 메시 캐시와 머티리얼

`MeshCache`는 초기화 시 `assets/models/` 디렉토리의 모든 `.obj` 파일을 자동 로드하고, 캐릭터(Box 0.4×0.9×0.4)와 발사체(Sphere 반지름 0.5)를 프로시저럴 메시로 추가 생성합니다. `ObjLoader`는 Wavefront OBJ의 정점 위치, UV, 법선을 파싱하며, 면 인덱싱 중복을 자동 제거합니다.

Kenney 3D 던전 에셋의 OBJ 파일은 표준 Wavefront OBJ 사양에 없는 정점 컬러 값을 `v` 라인 뒤쪽에 추가로 붙여 두는 비표준 확장을 사용합니다. 정점 위치는 `std::istringstream`으로 앞 세 float만 읽어 뒤따르는 컬러 값이 자연스럽게 무시되도록 파서를 작성했습니다. 더 큰 문제는 일부 face가 삼각형이 아닌 N-gon(4개 이상 정점)으로 구성된 점이었는데, 삼각형 가정의 인덱스 버퍼 생성이 깨지므로 face 라인의 정점 토큰을 모두 읽은 뒤 `(0, i-1, i)` 패턴의 triangle fan으로 분할하는 처리를 넣어 해결했습니다. OBJ에 법선이 포함되지 않은 경우에는 face normal을 cross product로 계산해 정점별로 누적·정규화하는 방식으로 smooth shading을 적용합니다.

`MaterialManager`는 OBJ의 MTL 파일에서 텍스처 경로를 읽어 `ID3D11ShaderResourceView`를 생성하고, PS의 `t0` 슬롯에 바인딩합니다. MTL이 없거나 텍스처 로드에 실패하면 마젠타색 fallback 텍스처가 적용됩니다.

로드된 모델 (29종): 캐릭터 3종(human, orc, character-a), 던전 구조 3종(floor, floor-detail, wall), 소품 23종(barrel, banner, chest, column, coin, dirt, gate, rocks, stairs, stones, torch, trap, wall-half, wall-narrow, wall-opening, wood-structure, wood-support 등 — Kenney 3D 던전 에셋)

4 공간

4.1 카메라

정사영(Orthographic) 투영으로 아이소메트릭 시점을 구현합니다. 투영 행렬은 `Width = orthoSize × aspect × 2`, `Height = orthoSize × 2`, Z범위 [-200, 500]으로 구성됩니다. Pitch 30°, Yaw 45°의 고정 각도에서 타겟 위치를 추적하며, Smoothing을 적용해 카메라를 자연스럽게 이동시킵니다. 빌보드 이펙트를 위해 매 프레임 카메라 Right/Up 벡터를 재계산합니다.

표 8: 카메라 파라미터

파라미터	값	설명
Pitch	30°	수직 관찰 각도
Yaw	45°	수평 회전 각도
Distance	20	카메라-타겟 거리
OrthoSize	15 (기본), 5~40 (줌 범위)	정사영 반높이
SmoothTime	0.15s	추적 감쇠 시간
ZoomSpeed	2.0	스크롤 줌 속도

카메라 위치는 다음 공식으로 결정됩니다.

```
카메라 위치 = 타겟 + (d·cos(pitch)·sin(yaw), d·sin(pitch), -d·cos(pitch)·cos(yaw))
```

매 프레임 `t = dt / kSmoothTime(0.15s)`로 보간 계수를 계산하고, 현재 위치를 타겟을 향해 `pos += (target - pos) * t`로 이동시킵니다. 초기 진입 시에는 `SnapTo()`로 즉시 이동합니다.

마우스 스크롤 델타에 `kZoomSpeed`(2.0)를 곱해 orthoSize를 조정하며, [5, 40] 범위로 클램프해 극단적 확대/축소를 방지합니다.

4.2 ScreenToGround 역투영

마우스 클릭 위치를 월드 좌표로 변환해 공격 방향을 결정해야 합니다. 정사영 카메라에서 스크린 좌표를 지면 교차점으로 변환하는 역투영 알고리즘을 구현했습니다.

스크린 좌표 (x, y)를 NDC로 변환합니다: `ndcX = 2x/w - 1`, `ndcY = 1 - 2y/h`.

NDC near(z=0)과 far(z=1) 좌표를 `invViewProj` 행렬로 월드 공간에 역투영해 두 점을 얻습니다.

near→far 방향의 레이와 y=groundY 수평면의 교차점 `t = (groundY - originY) / dirY`를 계산합니다.

`dirY ≈ 0`인 경우(시선과 지면이 거의 평행) `t = 0`으로 폴백합니다.

Camera.cpp — ScreenToGround()

```
XMFLOAT3 Camera::ScreenToGround(float mouseX, float mouseY,
                                float screenW, float screenH, float groundY) const
{
    float ndcX = 2.0f * mouseX / screenW - 1.0f;
    float ndcY = 1.0f - 2.0f * mouseY / screenH;

    XMMATRIX invVP = XMMatrixInverse(nullptr, GetViewProj());
```

```
    XMVECTOR nearPt = XMVector3TransformCoord(
        XMVectorSet(ndcX, ndcY, 0.0f, 1.0f), invVP);
    XMVECTOR farPt  = XMVector3TransformCoord(
        XMVectorSet(ndcX, ndcY, 1.0f, 1.0f), invVP);

    XMVECTOR rayDir = XMVectorSubtract(farPt, nearPt);
    float originY = XMVectorGetY(nearPt);
    float dirY = XMVectorGetY(rayDir);
    float t = (dirY != 0.0f) ? (groundY - originY) / dirY : 0.0f;

    XMVECTOR hit = XMVectorAdd(nearPt, XMVectorScale(rayDir, t));
    XMFLOAT3 result;
    XMStoreFloat3(&result, hit);
    return result;
}
```

4.3 이동 송신 최적화

이동 속도는 Walk 5.0 m/s, Run(Shift) 8.0 m/s이며, 20Hz(50ms 간격)로 서버에 위치를 동기화합니다. 위치 변화 제곱거리가 0.0001 미만이고 회전 변화가 0.5° 미만이면 전송을 건너뛰어 정지 상태의 불필요한 트래픽을 차단합니다.

PlayerController.cpp — 20Hz 전송 최적화

```
sendTimer_ += dt;
if (sendTimer_ >= kSendRate) {    // kSendRate = 0.05s
    sendTimer_ -= kSendRate;
    float distSq = dx*dx + dy*dy + dz*dz;
    float rotDiff = fabsf(data_->rotationY - lastSentRot_);
    if (distSq > 0.0001f || rotDiff > 0.5f) {
        sendPending_ = true;
        lastSentPos_ = data_->position;
        lastSentRot_ = data_->rotationY;
    }
}
```

5 네트워크 아키텍처

5.1 서버 주도 흐름 제어 구현

두 가지 씬 전환 방법을 고민했습니다. 클라이언트가 패킷을 직접 버퍼링했다가 씬 전환 후 드레인하는 방법, 그리고 서버가 브로드캐스트를 보류하는 방법입니다. 전자는 씬마다 “어디까지 버퍼링하고 언제 실시간으로 전환할지”를 판단해야 해서 같은 로직이 반복되고, 전환 도중 도착한 패킷이 다음 씬의 핸들러 등록 전에 파싱되어 상태가 어긋날 여지가 있었습니다.

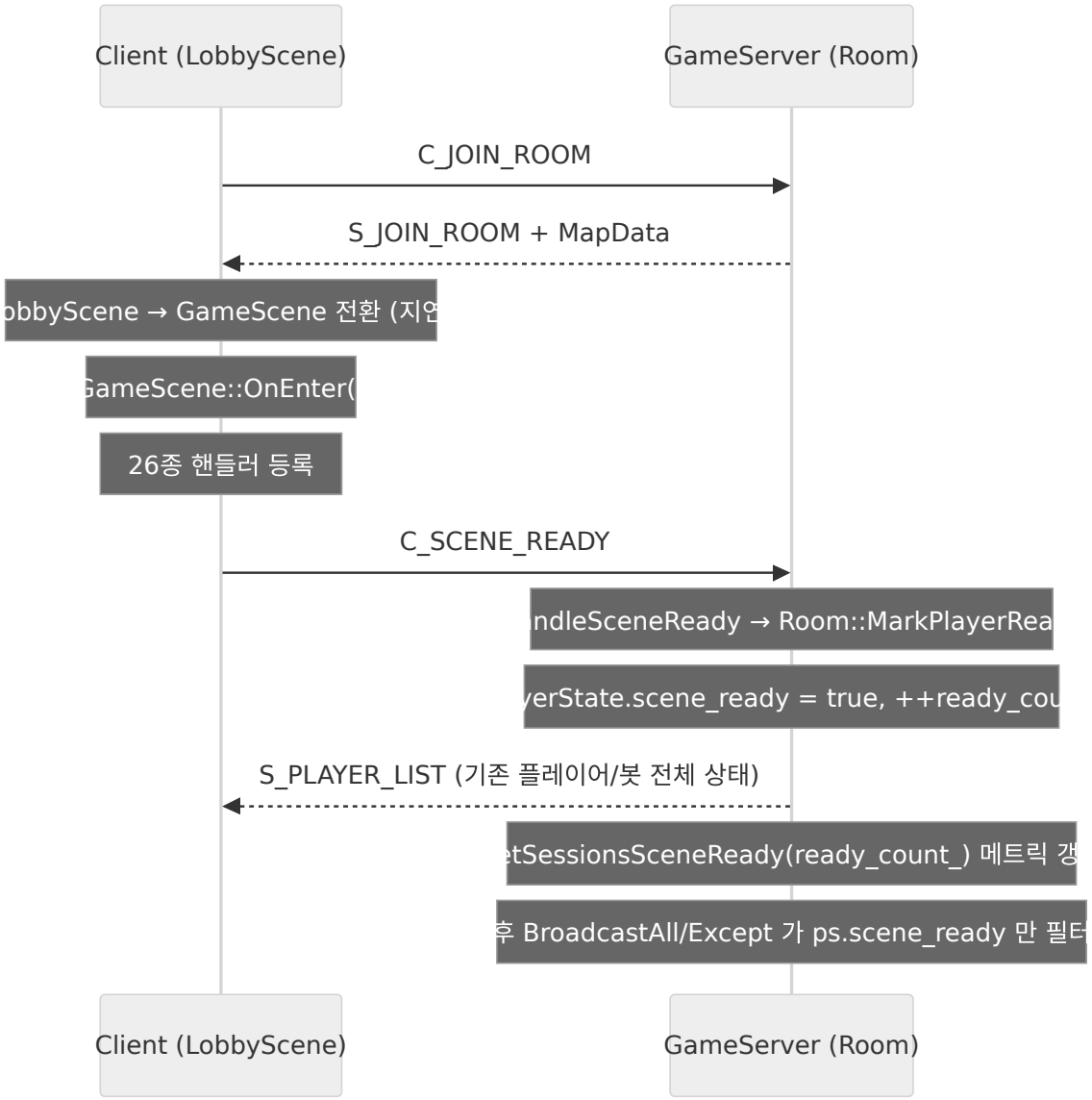

그림 5: 서버 주도 흐름 제어 흐름

서버가 `C_SCENE_READY` 수신 전까지 해당 플레이어로 가는 브로드캐스트를 필터링하면, 클라이언트는 준비 완료를 한 번 알리기만 하면 되고 버퍼링·드레인 로직 자체가 사라집니다. 서버 측 브로드캐스트 경로는 이미 `Room::BroadcastAll` / `BroadcastExcept` 로 중앙집중화되어 있어, `PlayerState::scene_ready` 플래그 하나를 기존 루프에 끼워넣는 것만으로 모든 송신 사이트가 자동으로 수혜를 받습니다. 플레이어별로 상태를 추적하는 이유는, 한 명의 느린 씬 전환이 같은 룸의 다른 플레이어를 블록하지 않도록 하기 위함입니다.

초기 구현은 준비 완료 플레이어를 별도 `std::unordered_set<PlayerId>` 에 담는 방식이었지만, 브로드캐스트 루프가 어차피 `players_` 맵을 순회하므로 같은 순회 안에서 `ps.scene_ready` 를 확인하면 별도 해시 lookup 없이 분기가 끝나고, “방에 있는가”와 “수신 가능한가”를 단일 `PlayerState` 로 통합해 두 컨테이너 간 동기화 문제와 ID 재사용 race를 함께 없앨 수 있었습니다.

6 게임 시스템

6.1 전투 시스템

모든 데미지 판정은 서버에서 수행하며, 클라이언트는 시각 효과만 담당합니다. 전투는 두 가지 경로로 나뉩니다.

발사체 공격 (좌클릭)

좌클릭 시 0.4초 쿨다운을 확인한 뒤 `ScreenToGround`로 조준 방향을 계산합니다. 플레이어 위치에서 조준 방향으로 0.6 오프셋된 지점(y+0.9)에 로컬 탄환을 즉시 스폰하고 `C_FIRE` 패킷을 전송합니다. 로컬 즉시 스폰으로 시각적 반응성을 확보하면서, 실제 히트 판정은 서버 `ProjectileManager`가 수행합니다.

`CombatManager`는 4종의 전투 시각 요소를 관리합니다.

표 9: 전투 효과 요소

요소	구현	수명	물리
Projectile	80 m/s 직진 + `IsWalkable()` 벽 충돌	traveled ≥ 50m 또는 벽 충돌 시 소멸	위치 누적 이동
AttackEffect	skillId별 이펙트 (Slash 셰이더)	0.3s (근접) / 0.6s (힐)	스케일 1.0→1.5 ease-out 감쇠
DamagePopup	ImGui 텍스트 (월드→스크린 투영)	1.5s	y += 2.0·dt 상승 + 알파 감쇠
HitEffect	타겟 위치 확장 링 (Hit 셰이더)	0.3s	y+0.9 오프셋

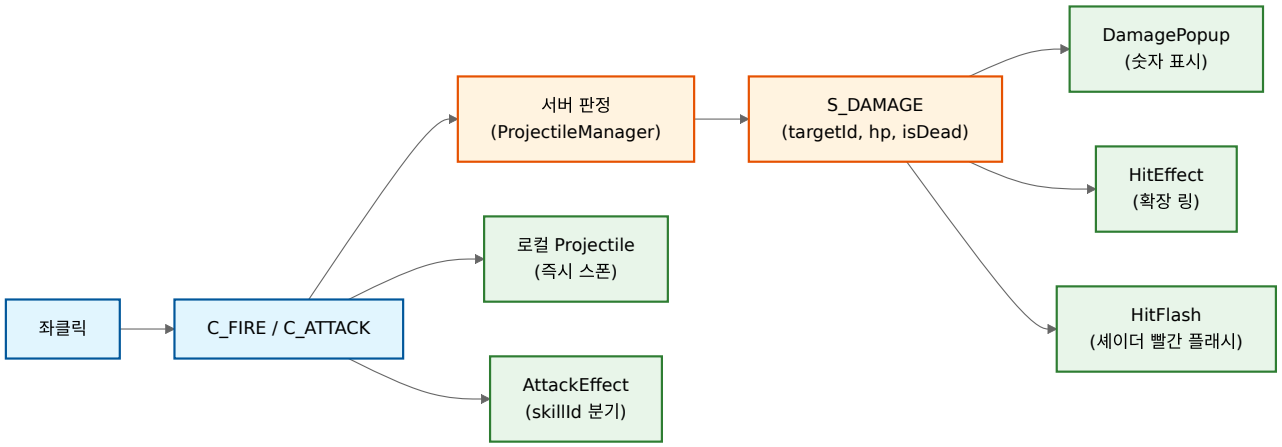

그림 6: 전투 이벤트 처리 흐름

6.2 던전 생성

`DungeonGenerator`는 서버에서 수신한 `MapData`를 파싱하여 던전 메시 인스턴스를 생성합니다. 레이아웃(grid), props(barrel/chest/rocks/banner/trap), lights, portals는 모두 서버가 이미 결정한 상태로 내려오고, 클라이언트는 각 항목을 순회하면서 해당 메시와 world matrix를 만들어 `InstanceRenderer`에 넘기기만 합니다. 바닥 타일의 `floor-detail` variant 선택과 90° 회전만 클라이언트에서 셀 좌표 해시로 정해, 동일한 `BuildFromMapData()` 호출이 항상 같은 결과를 내도록 했습니다.

표 10: 그리드 셀 해석

셀 값	의미	처리
0	바닥	`floor` / `floor-detail` 메시 + 90° 단위 회전
1	벽	인접 4방향 검사 → 바닥이 있는 방향으로 회전

벽은 인접 셀만 보면 회전이 결정되므로 클라이언트마다 자연스럽게 일치합니다. 초기 구현에서 벽 메시를 회전 없이 배치했을 때 face normal이 플레이어 반대쪽을 향해 backface culling으로 벽이 보이지 않는 현상이 있었고, 인접한 바닥 셀 방향으로 회전시켜(North=0°, East=90°, South=180°, West=270°) 정면이 항상 통로를 향하도록 했습니다. 사방이 모두 벽이면 내부 벽으로 간주해 스폰 자체를 생략합니다.

DungeonGenerator.cpp — 벽 인접성과 회전

```
bool adjN = IsFloor(grid, gw, gh, x, z - 1);
bool adjS = IsFloor(grid, gw, gh, x, z + 1);
bool adjE = IsFloor(grid, gw, gh, x + 1, z);
bool adjW = IsFloor(grid, gw, gh, x - 1, z);

if (adjN || adjS || adjE || adjW) {
    float rotY = 0.0f;
    if (adjN)      rotY = 0.0f;
    else if (adjE) rotY = 90.0f;
    else if (adjS) rotY = 180.0f;
    else if (adjW) rotY = 270.0f;
    // ... wall 메시 스폰
}
```

서버 `MapData`의 `props` 배열에서 5종(barrel, chest, rocks, banner, trap)의 위치와 `rotation_y`를 그대로 읽어 World Matrix를 구성합니다. `MapLight`에서는 위치(y+1.5 오프셋), RGB, intensity, range를 추출해 포인트 라이트 배열로 셰이더에 전달하고, `MapPortal`에서는 위치·portal_id·target_name을 읽어 포털 엔티티를 생성합니다. 서버가 `S_JOIN_ROOM` 응답에 `MapData`를 포함해 전송하면, 클라이언트는 `BuildFromMapData()` 한 번으로 모든 인스턴스 목록을 만들어 `InstanceRenderer`에 넘기는 것으로 던전 로딩이 끝납니다.

6.3 원격 엔티티 보간

`EntityManager`는 서버에서 수신한 원격 플레이어의 위치를 즉시 적용하지 않고, `targetPos` / `targetRotY`에 저장한 뒤 매 프레임 선형 보간(Lerp)합니다. 20Hz 네트워크 업데이트를 60fps 렌더링으로 부드럽게 표현합니다.

회전 보간에서 단순히 `current + (target - current) * t`를 적용하면, 359° → 1° 전환 시 차이가 -358°로 계산되어 거의 한 바퀴를 역회전하는 문제가 발생합니다. 실제로는 2°만 정방향으로 회전해야 하므로, 차이를 `[-180°, +180°]` 범위로 정규화해 항상 최단 경로로 회전하도록 했습니다.

EntityManager.cpp — 보간과 히트 타이머

```
float t = std::min(dt * kLerpSpeed, 1.0f);  // kLerpSpeed = 10.0
for (auto& [id, e] : entities_) {
    e.data.position.x += (e.targetPos.x - e.data.position.x) * t;
    e.data.position.z += (e.targetPos.z - e.data.position.z) * t;

    if (e.hitTimer > 0) e.hitTimer -= dt;

    // 회전 보간: ±180° 정규화로 최단 경로 회전
    float diff = e.targetRotY - e.data.rotationY;
    while (diff > 180.0f) diff -= 360.0f;
    while (diff < -180.0f) diff += 360.0f;
    e.data.rotationY += diff * t;
}
```

`isDead` 플래그가 설정된 엔티티는 렌더링에서 제외하고, `hitTimer`가 0보다 크면 `HitFlash` 파라미터로 전달해 피격 효과를 표시합니다. `moveState`(0=idle, 1=walk, 2=run)에 따라 메시 스케일 등 시각적 피드백을 조정합니다.

7 확장 기능

7.1 미니맵 셰이더

우측 상단에 `minimap.hlsl` 전용 셰이더로 던전 전체 지도를 렌더링합니다. 서버 그리드를 `R8_UNORM` 텍스처로 업로드하고, 플레이어 중심 시점으로 `ViewRadius`만큼 표시합니다. Vertex Shader에서 `SV_VertexID`만으로 쿼드를 생성하므로 정점 버퍼가 불필요합니다.

로컬 플레이어(하늘색)와 원격 플레이어(빨간 점)를 `EntityPositions[16]` 상수 배열로 전달해 셰이더에서 직접 렌더링합니다. 가장자리 `smoothstep` 페이드아웃과 알파 블렌딩으로 게임 화면 위에 반투명하게 오버레이됩니다.

7.2 LoL 스타일 에임 라인

플레이어 위치에서 마우스 에임 방향으로 8유닛 길이의 지면 라인을 16세그먼트로 월드→스크린 투영합니다. `DungeonGenerator::IsWalkable()` 로 벽 충돌을 체크해 벽에서 라인이 끊기며, 거리에 따라 알파가 감쇠합니다. 에임 지점에는 정사영 투영된 타원형 조준원이 표시됩니다.

7.3 벽 충돌 시스템

클라이언트와 서버 양쪽에서 벽 충돌을 검증합니다.

표 11: 벽 충돌 검증 레이어

레이어	구현	목적
클라이언트 `PlayerController`	`IsWalkable()` + 축 분리 슬라이딩	즉각적 반응 UX
클라이언트 `CombatManager`	탄환 매 프레임 `IsWalkable()` 체크	벽 관통 탄환 방지
서버 `HandleMove()`	`GetTile()` 체크, 벽이면 이동 거부	치트 방지

초기 벽 충돌 구현에서는 이동 목표가 벽이면 단순히 이동을 차단했습니다. W+D(대각선)로 벽을 향해 달리면 벽 앞에서 완전히 정지하여, 벽을 따라 이동하려면 정확히 한 방향 키만 눌러야 하는 답답한 조작감이 생겼습니다. `PlayerController` 는 3단계 폴백 축 분리(Axis-Separated) 슬라이딩으로 이를 해결합니다. 대각선 이동이 벽에 막히면 X축 단독, Z축 단독 순서로 시도해 이동 가능한 축으로 미끄러집니다.

PlayerController.cpp — 축 분리 슬라이딩

```
float newX = data_->position.x + moveDir.x * speed * dt;
float newZ = data_->position.z + moveDir.z * speed * dt;

if (dungeon_) {
    // 1. 전체 이동 시도
    if (dungeon_->IsWalkable(newX, newZ)) {
        data_->position.x = newX;
        data_->position.z = newZ;
    }
    // 2. X축 단독 슬라이드
    else if (dungeon_->IsWalkable(newX, data_->position.z)) {
        data_->position.x = newX;
    }
    // 3. Z축 단독 슬라이드
    else if (dungeon_->IsWalkable(data_->position.x, newZ)) {
        data_->position.z = newZ;
    }
    // 4. 양축 모두 차단 — 정지
}
```

7.4 킬/데스 리더보드

서버가 `PlayerState` 에 `kills` / `deaths` 카운터를 관리하고, Room 틱에서 5초마다 `S_SCOREBOARD` 를 브로드캐스트합니다. 클라이언트는 Tab 키를 누르고 있을 때 화면 중앙에 킬 수 내림차순으로 정렬된 리더보드를 표시하며, 로컬 플레이어를 하늘색으로 강조합니다.

7.5 킬피드 + 이름표

사망 이벤트(`S_DAMAGE`의 `is_dead=true`) 발생 시 우측에 “A > B” 형식의 킬피드를 5초간 표시합니다. 최대 5개까지 스택되며 0.5초에 걸쳐 페이드 아웃됩니다.

원격 플레이어의 이름은 항상 머리 위에 표시되며, HP바는 피격 후 3초간만 나타납니다. `hpBarTimer`와 `hitFlashTimer`를 분리해 셰이더 피격 플래시(0.25초)와 UI HP바 표시(3초)가 독립적으로 동작합니다.

7.6 FBX 로딩 (Assimp)

`MeshCache`가 Assimp 라이브러리(`aiProcess_Triangulate | aiProcess_GenSmoothNormals`)로 `assets/fbx/` 디렉토리의 FBX 파일을 자동 로드합니다. OBJ가 유효한 메시(인덱스 > 0)면 OBJ를 우선하고, placeholder(빈 파일)면 FBX를 사용합니다. FBX 로드 시 모든 서브메시를 합치고, 바운딩 박스 높이를 `uniformScale = 2.7 / (maxY - minY)`로 정규화하며, UV의 V 좌표를 `1.0 - v`로 플립합니다.

7.7 그래프 기반 존 전환 (포탈)

Room 간 연결을 방향 없는 그래프로 관리합니다. 포탈 진입 시 서버가 양방향 링크를 생성하므로 순환 경로(R1→R2→R3→R1)가 가능합니다.

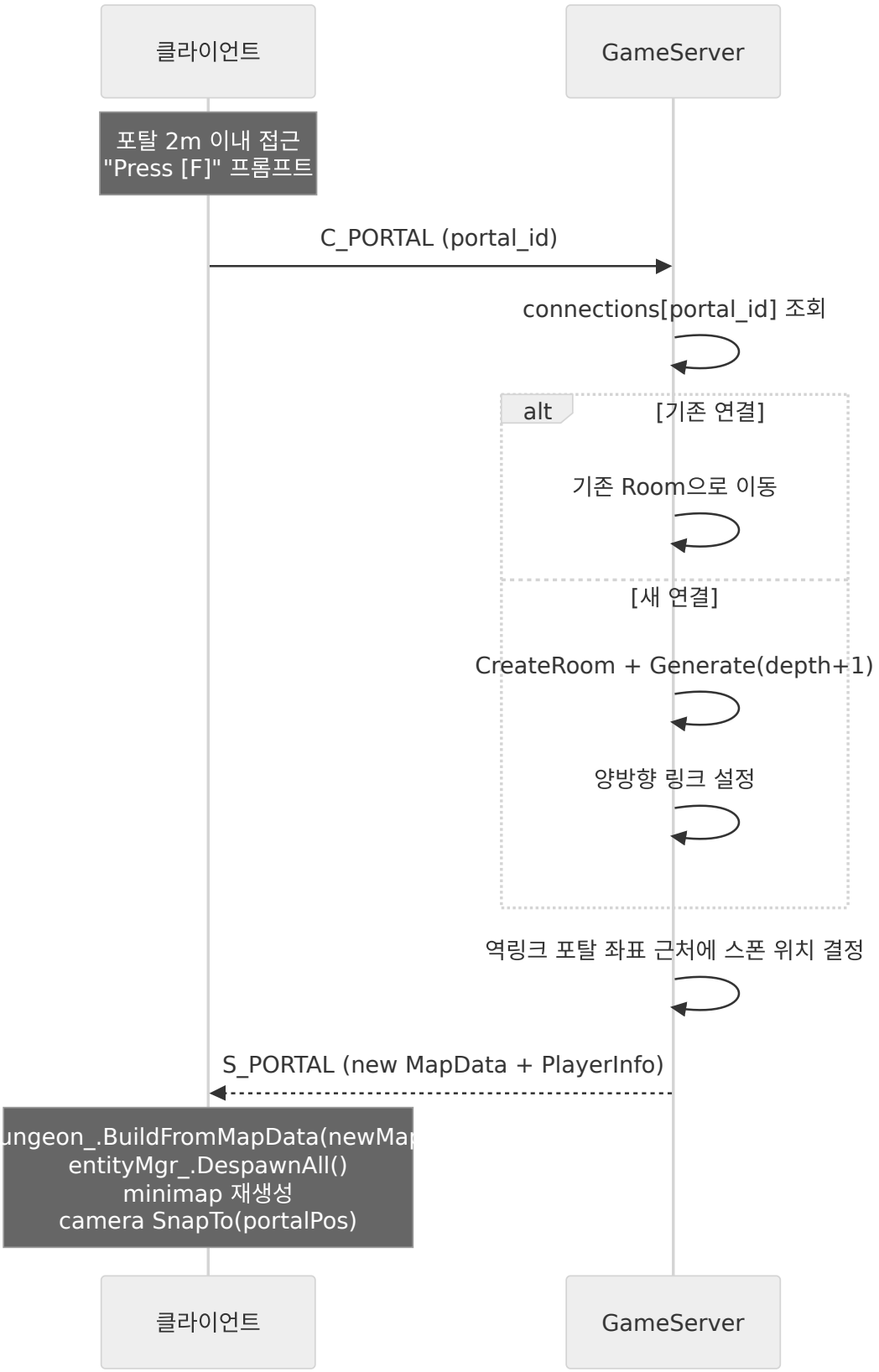

그림 7: 포탈 존 전환 흐름

클라이언트는 씬 전환 없이 `GameScene` 내부 상태만 리셋합니다. 포탈 위치에 column 메시를 렌더링하고, 미니맵에 노란 점으로 표시합니다. Zone depth 정보를 추적해 방문 기록을 유지합니다.

7.8 전체 지도 (M키)

M키로 토글하는 전체 던전 지도를 화면 중앙에 오버레이합니다. `ImGui::GetForegroundDrawList()` 로 렌더링하며, 각 그리드 셀을 4px 사각형으로 표시합니다.

표 12: 전체 지도 요소

요소	표시	색상
벽	어두운 사각형	(60, 50, 70)
바닥	밝은 사각형	(140, 130, 150)
전진 포탈	사각형 + 라벨	노란색
후진 포탈	사각형 + 라벨	하늘색
로컬 플레이어	원(r=4)	하늘색
원격 엔티티	원(r=2.5)	빨간색
바닥 아이템	원(r=2)	주황색

7.9 인벤토리 목록형 UI

10×10 그리드에서 목록형으로 전환하여 빈 슬롯을 숨기고, 보유 아이템만 `ImGui::Selectable` 로 표시합니다. 독텍(Dog Tag)은 `GetItemName()` 에서 전용 이름을 반환하며, 우클릭 컨텍스트 메뉴로 Use/Drop이 가능합니다.

7.10 로그인 흐름 개선

TCP 접속과 계정 로그인을 분리했습니다.

1. 로그인 화면에 서버 주소/포트 입력 필드 표시
2. Connect 버튼으로 TCP 연결 → 연결 성공 후 Login/Register 버튼 활성화
3. `TcpClient::Connect()` 에서 `getaddrinfo()` 로 도메인 이름 resolve 지원
4. 연결 실패 시 `lastError_` 에 원인 저장 → UI에 표시

이전에는 Login 버튼이 접속+로그인을 동시에 시도해 첫 접속 시 패킷이 유실되는 타이밍 문제가 있었습니다.

7.11 독텍 루팅 시스템

적 사망 시 사망 위치에 독텍이 스폰되어 10m 이내에서 라벨이 표시되고, 3m 이내에서 “Press [E]” 프롬프트가 나타납니다. E키 입력 시 `C_PICKUP` 전송 → 서버 거리 검증 → `S_ITEM_ADD` 로 인벤토리 추가 + `S_GROUND_ITEM_DESPAWN` 으로 모든 클라이언트에서 제거됩니다.

바닥 아이템은 coin 메시로 렌더링되며, 최대 5개까지만 라벨을 표시해 성능을 유지합니다. 사망 중에는 프롬프트가 숨겨집니다.

7.12 사망 드롭 + 재화

사망 시 서버가 공격자에게 골드 10~50을 즉시 보상(`S_CURRENCY_UPDATE`)하고, 30% 확률로 랜덤 아이템을 지급(`S_ITEM_ADD`)합니다. 재화는 Gold만 유지하며, HP바 아래에 표시됩니다. 독텍 아이템 사용 시 +50 Gold 보상입니다.

7.13 GameScene UI 컴포넌트 구성

`GameScene::OnUI()` 는 13개의 전용 드로우 함수로 분리되어 각 UI 요소의 표시 조건과 렌더링을 독립적으로 관리합니다.

표 13: GameScene UI 컴포넌트

함수	표시 조건	역할
DrawHPBar	항상	HP·레벨·골드, 스킬 쿨다운
DrawCrosshair	항상	LoL 스타일 에임 라인 + 조준원
DrawMinimap	항상	셰이더 기반 우측 상단 미니맵
DrawRemoteHPBars	피격 후 3s	원격 플레이어 이름표 + HP바
DrawDamagePopups	피격 시 1.5s	부유 데미지 숫자
DrawChat	항상	채널별 채팅 (All/Party/Whisper)
DrawInventory	I키 토글	목록형 아이템 UI + 컨텍스트 메뉴
DrawPartyPanel	파티 참가 시	파티원 HP·레벨
DrawLeaderboard	Tab 누르는 동안	K/D 리더보드
DrawKillFeed	킬 발생 시 5s	우측 킬피드 스택
DrawFullMap	M키 토글	전체 던전 지도 오버레이
DrawDeathOverlay	사망 시	리스폰 카운트다운
DrawExitMenu	ESC 토글	방 나가기 메뉴

`WndProc`에서 `ImGui::GetIO().WantTextInput`과 `WantCaptureMouse`를 체크하여 게임 입력과 UI 입력을 분리합니다. 채팅 입력 중에는 키보드 이벤트가 게임으로 전달되지 않고, 인벤토리 위에서는 마우스 클릭이 게임 입력으로 처리되지 않습니다. 마우스 UP 이벤트는 항상 게임에 전달하여 버튼이 눌린 채 고착되는 현상을 방지합니다.

7.14 ImGui 윈도우 포커스 수정

ImGui는 윈도우를 생성하면 자동으로 해당 윈도우에 포커스를 부여합니다. 데미지 팝업은 피격 시마다 `ImGui::Begin()`으로 생성되는데, 인벤토리(I키)를 열어둔 상태에서 피격되면 팝업이 매 프레임 포커스를 가져가면서 인벤토리의 `Selectable` 클릭이 무시되는 버그가 발생했습니다. 원인은 ImGui의 윈도우 생성 시 기본 동작인 자동 포커스 이동이었습니다. HP바, 채팅, 리더보드, 파티, 데미지팝업 등 상호작용이 불필요한 모든 HUD 윈도우에 `ImGuiWindowFlags_NoBringToFrontOnFocus | ImGuiWindowFlags_NoFocusOnAppearing` 플래그를 적용해, 포커스를 가져가지 않도록 했습니다.

7.15 서버 포트폴리오 연관성

표 14: 클라이언트-서버 연관 기술

클라이언트 요소	서버 요소	연관 기술
TcpClient + PacketFramer	Session + RecvBuffer	Binary Header + Protobuf 프로토콜
DungeonGenerator.BuildFromMapData()	DungeonGenerator (서버)	MapData 직렬화·동기화
C_FIRE → 로컬 스폰 → S_DAMAGE	ProjectileManager	클라이언트 예측 + 서버 권위적 판정
EntityManager 보간	Room 20Hz 브로드캐스트	이동 동기화
HandleMove 벽 충돌 거부	DungeonGenerator.GetTile()	서버 권위적 이동 검증

클라이언트 요소	서버 요소	연관 기술
S_SCOREBOARD → 리더보드	PlayerState kills/deaths	서버 권위적 킬/데스 추적
C_PORTAL → 그래프 존 전환	Room.Connections() 양방향 링크	그래프 기반 Room 이동
S_GROUND_ITEM_SPAWN → E키 줍기	Room.SpawnGroundItem()	독텍 루팅 시스템
S_CURRENCY_UPDATE + S_ITEM_ADD	GrantKillReward()	사망 보상 + DB 영속화
DrawFullMap (M키)	MapData 그리드 + 포탈	전체 지도 시각화
getaddrinfo() 도메인 접속	Listener 0.0.0.0:7777	원격 서버 접속

서버 아키텍처 상세(io_uring 네트워크 엔진, JobQueue 공정 스케줄링, BufferPool, 벤치마크 등)는 ServerCore 포트폴리오를 참조하시기 바랍니다.